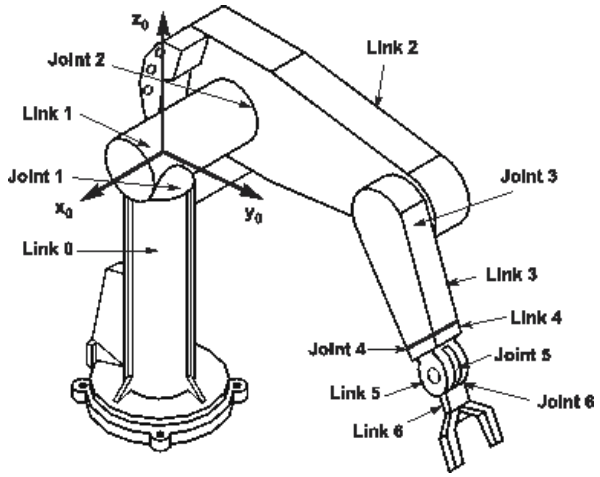
Lecture 9: Velocity Propagation & Trajectory Generation

Iñigo Iturrate
Associate Professor
SDU Robotics,
The Maersk McKinney Moller Institute,
University of Southern Denmark

Ø27-604-3
inju@mmmi.sdu.dk



SDU ROBOTICS

What are we doing today?

1. Introduction to Robotics & Recap of Linear Algebra and Mathematical Notation
2. Translations & Rotation Matrices
3. Other Representations for Orientation
4. Transformation Matrices
5. Forward Kinematics & DH Parameters
6. Inverse Kinematics

7. Kinematic Simulation
8. Schematic Representations of Robots, Modeling a Real Robot

9. **Angular Velocities, Velocity Propagation & Trajectory Generation** ***(Today)***
10. The Jacobian Matrix, Singularities
11. Analytical & Geometrical Jacobian, Numerical IK, Statics
12. Manipulability, More on the Robotic Systems Toolbox



"Robots that stay still",
i.e. positions/orientations

Simulation

"Robots that move",
i.e. velocities

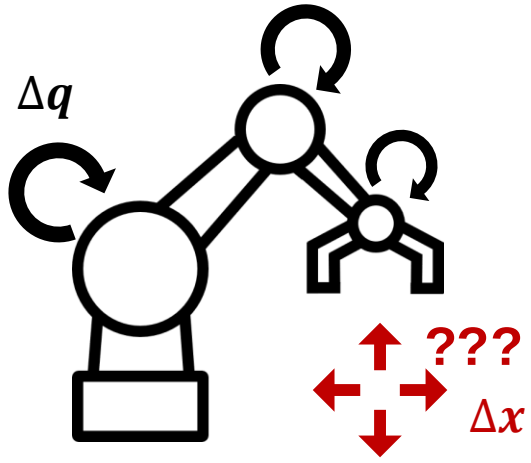
SDU ROBOTICS

Today…

So far, we have worked with **robots at standstill** and described their **positions**,
both in **joint** and **Cartesian space**.

Often, however, we will want to consider **velocities** when **describing motion**.

Particularly, we might want to see how a **small change of joint angles** influences the **change in end-effector position**, and vice versa.



Topics for Today

Part I: Linear & Angular Velocities

- Linear and Angular velocities
- Derivative of a rotation matrix
- Other representations of angular velocity
- Velocity of Links

Part II: Trajectory Generation

- Point-to-point (PTP) movements
  - In Joint space and Cartesian space (position only!)
- Trapezoidal velocity profiles
- Quadratic, Cubic and Higher-order Polynomials
- Chaining multiple PTP movements

Part I: Linear & Angular Velocities

Velocity Vectors as Differentiated Position

If we have a **point,** ${}^{B}\boldsymbol{Q}$, seen from frame {B}, we can **define velocity as a difference in positions**:

$$ {}^{B}\boldsymbol{V}_{Q} = \frac{d}{dt}\,{}^{B}Q = \lim_{\Delta t \to 0} \frac{{}^{B}Q(t+\Delta t) - {}^{B}Q(t)}{\Delta t} $$

Or as the **time-derivative of position**:

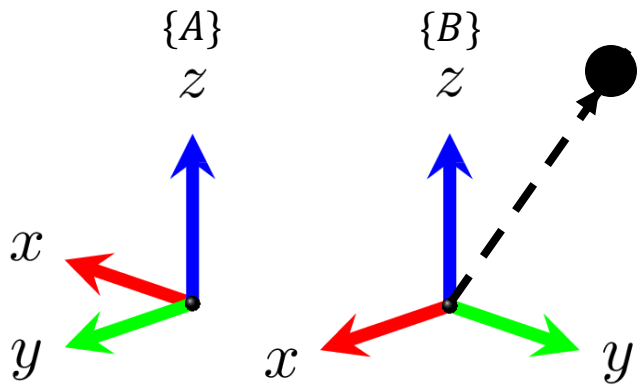
Notice that the velocity depends on the reference frame!

$$ {}^{A}\left({}^{B}\boldsymbol{V}_{Q}\right) = \frac{{}^{A}d}{dt}\,{}^{B}Q(t) $$

But also on the frame it is expressed in (because {B} could move with respect to {A})

We can therefore also write:

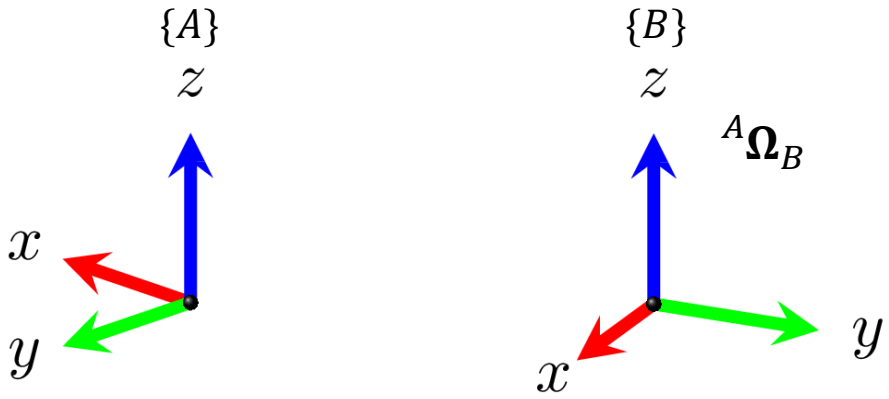
$$ {}^{A}\left({}^{B}\boldsymbol{V}_{Q}\right) = {}^{A}_{B}\boldsymbol{R}\,{}^{B}\boldsymbol{V}_{Q} $$



SDU ROBOTICS

Angular Velocity Vectors

Angular velocity, ${}^{A}\boldsymbol{\Omega}_{B}$ (or ${}^{A}\boldsymbol{\omega}_{B}$), describes **how a frame rotates relative to another frame**:



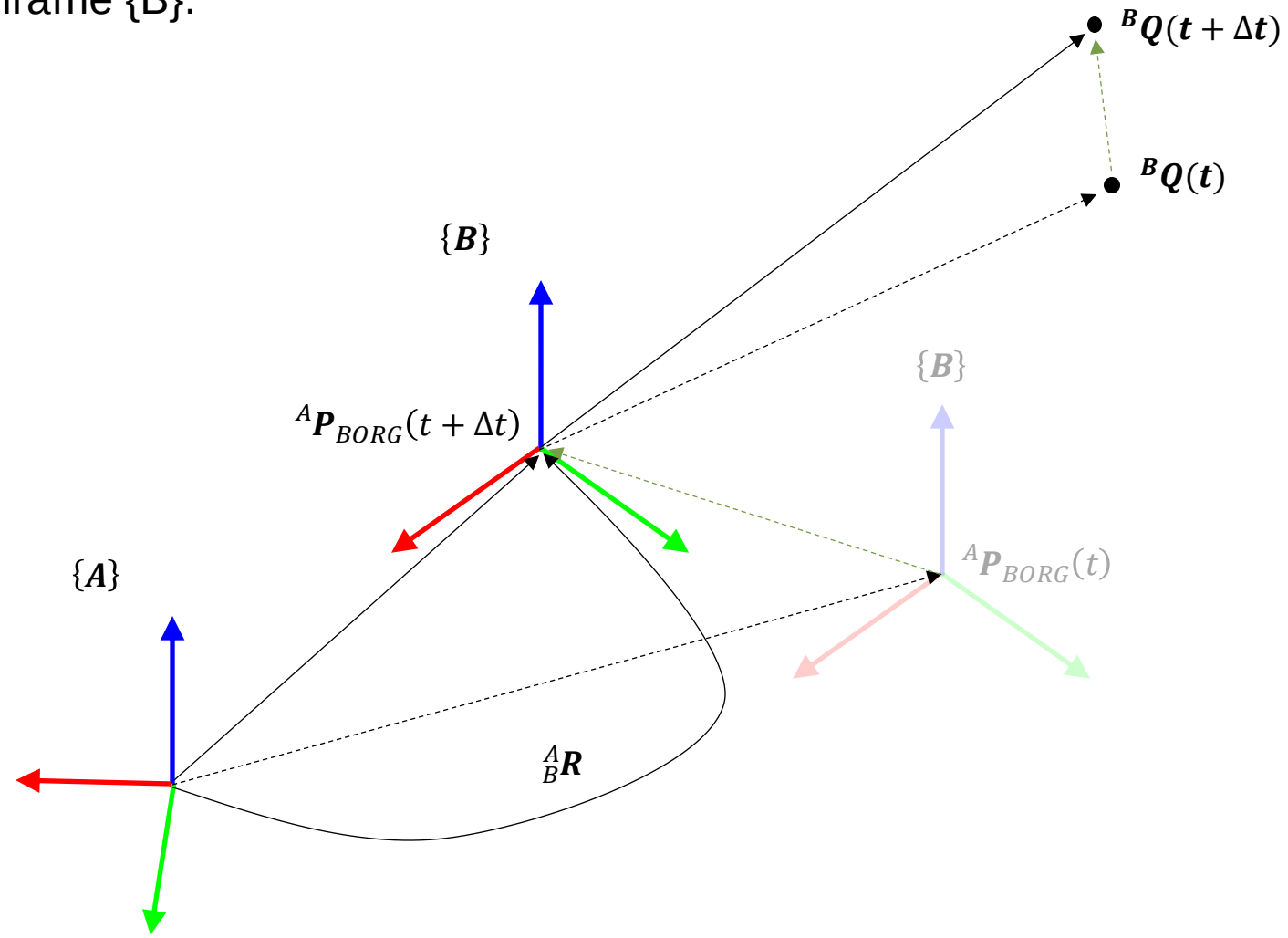
It is made up of a **unit vector** describing an **axis of rotation** and a **magnitude** describing a **speed of rotation:**

$$\boldsymbol{\Omega} = \begin{bmatrix} \Omega_x \\ \Omega_y \\ \Omega_z \end{bmatrix} = \Omega \cdot \hat{\boldsymbol{u}} = \Omega \cdot \begin{bmatrix} u_x \\ u_y \\ u_z \end{bmatrix}$$

Think of it as axis-angle, but where the axis and angle can change over time, i.e. $\Omega(t), \hat{\boldsymbol{u}}(t)$.

Linear Velocity of Rigid Bodies

Linear Velocity: Frame {B} moves translationally relative to frame {A}, but does not rotate. Point ${}^{B}\boldsymbol{Q}$ moves relative to frame {B}.



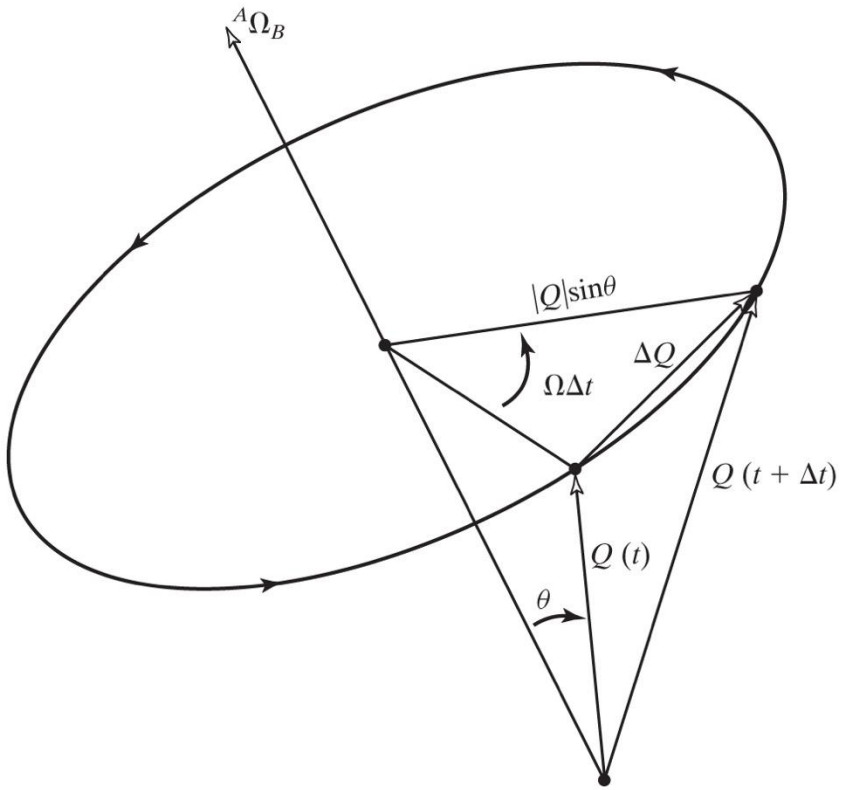
$$ {}^{A}\boldsymbol{V}_{Q} = {}^{A}\boldsymbol{V}_{BORG} + {}^{A}_{B}\boldsymbol{R}\,{}^{B}\boldsymbol{V}_{Q} $$

SDU ROBOTICS

Rotational Velocity of Rigid Bodies

Rotational Velocity of Frame: The orientation of frame {B} changes w.r.t. {A}.

Point $\boldsymbol{Q}$ is still relative to frame {B}.



Small-angle approximation:

If $\Delta t \to 0$ then $\sin \Omega\Delta t \approx \Omega\Delta t$

$$|\Delta \boldsymbol{Q}| = \left(\left|{}^{A}\boldsymbol{Q}\right| \sin\theta\right)\left(\left|{}^{A}\boldsymbol{\Omega}_{B}\right|\Delta t\right)$$

$$ {}^{A}\boldsymbol{V}_{Q} = {}^{A}\boldsymbol{\Omega}_{B} \times {}^{A}\boldsymbol{Q}$$

$$ = {}^{A}\boldsymbol{\Omega}_{B} \times {}^{A}_{B}\boldsymbol{R}\,{}^{B}\boldsymbol{Q}$$

From definition of cross-product

Linear & Rotational Velocity of Rigid Bodies

Mini-Exercise (discuss with your neighbors, 10 minutes)

1. **Rotational Velocity of Frame & Linear Velocity of Point**: The orientation of frame {B} changes w.r.t. {A}. Point $\boldsymbol{Q}$ moves relative to frame {B}.

$$^{A}\boldsymbol{V}_{Q} = {}^{A}_{B}\boldsymbol{R}\,{}^{B}\boldsymbol{V}_{Q} + {}^{A}\boldsymbol{\Omega}_{B} \times {}^{A}_{B}\boldsymbol{R}\,{}^{B}\boldsymbol{Q}$$

2. **General Case**: Frame {B} moves relative to frame {A} both translationally and rotationally. Point $\boldsymbol{Q}$ moves relative to frame {B}.

$$^{A}\boldsymbol{V}_{Q} = {}^{A}\boldsymbol{V}_{BORG} + {}^{A}_{B}\boldsymbol{R}\,{}^{B}\boldsymbol{V}_{Q} + {}^{A}\boldsymbol{\Omega}_{B} \times {}^{A}_{B}\boldsymbol{R}\,{}^{B}\boldsymbol{Q}$$

How do we relate rotation matrices to angular velocities?

Derivative of a Rotation Matrix

Consider a **rotation matrix that changes over time**, so $\boldsymbol{R} = \boldsymbol{R}(t)$.

Since a rotation matrix must be **orthogonal**: $\boldsymbol{R}(t)\boldsymbol{R}^T(t) = \boldsymbol{I}$

If we take the time-derivative:

$$\dot{\boldsymbol{R}}(t)\boldsymbol{R}^T(t) + \boldsymbol{R}(t)\dot{\boldsymbol{R}}^T(t) = \boldsymbol{0}$$

$$\dot{\boldsymbol{R}}(t)\boldsymbol{R}^T(t) + \left(\dot{\boldsymbol{R}}(t)\boldsymbol{R}^T(t)\right)^T = \boldsymbol{0}$$

If we define: $\boldsymbol{S}(t) = \dot{\boldsymbol{R}}(t)\boldsymbol{R}^T(t)$, then we have:

$$\boldsymbol{S}(t) + \boldsymbol{S}^T(t) = \boldsymbol{0}$$

$\boldsymbol{S}^T = -\boldsymbol{S}$ **(Skew-symmetric)**

We can also rearrange that to obtain: $\dot{\boldsymbol{R}}(t) = \boldsymbol{S}(t)\boldsymbol{R}(t)$

This means **we can relate the derivative of a rotation matrix to a skew-symmetric matrix, $\boldsymbol{S}(t)$.**

Skew-Symmetric Matrices

It so happens that **we can relate this skew-symmetric matrix to angular velocity** as well:

$$\boldsymbol{\Omega} = \begin{bmatrix} \Omega_x \\ \Omega_y \\ \Omega_z \end{bmatrix} \longleftrightarrow \boldsymbol{S}(\boldsymbol{\Omega}) = \begin{bmatrix} 0 & -\Omega_z & \Omega_y \\ \Omega_z & 0 & -\Omega_x \\ -\Omega_y & \Omega_x & 0 \end{bmatrix}$$

So, if we have a point in {B} that does not move, but {B} moves w.r.t. {A}:

$$^{A}\boldsymbol{P} = {}^{A}_{B}\boldsymbol{R}\ {}^{B}\boldsymbol{P}$$

$$^{A}\boldsymbol{V}_P = {}^{A}\dot{\boldsymbol{P}} = {}^{A}_{B}\dot{\boldsymbol{R}}\ {}^{B}\boldsymbol{P} \longleftarrow {}^{B}\boldsymbol{P} = {}^{B}_{A}\boldsymbol{R}\ {}^{A}\boldsymbol{P} = {}^{A}_{B}\boldsymbol{R}^T\ {}^{A}\boldsymbol{P}$$

$$^{A}\boldsymbol{V}_P = {}^{A}_{B}\dot{\boldsymbol{R}}\ {}^{A}_{B}\boldsymbol{R}^T\ {}^{A}\boldsymbol{P}$$

$$^{A}\boldsymbol{V}_P = {}^{A}_{B}\boldsymbol{S}\ {}^{A}\boldsymbol{P} \longleftrightarrow {}^{A}\boldsymbol{V}_P = {}^{A}\boldsymbol{\Omega}_B \times {}^{A}\boldsymbol{P}$$

Other Representations of Angular Velocity

We have seen we can represent **a change in the orientation over time** as:

- An **angular velocity** (closely related to **axis-angle** representation) → $\boldsymbol{\Omega}$
- A **screw-symmetric** matrix (the **derivative of a rotation matrix**) → $\boldsymbol{S}(\boldsymbol{\Omega}) = \dot{\boldsymbol{R}}\boldsymbol{R}^T$

***Question to you*: Does anyone feel that we are missing anything?**
(*Think back to Lecture 5)*

We can also represent derivatives of other orientation representations, e.g. Z-Y-Z Euler angles:

$$\dot{\boldsymbol{\Theta}}_{Z'Y'Z'} = \begin{bmatrix} \dot{\alpha} \\ \dot{\beta} \\ \dot{\gamma} \end{bmatrix}$$

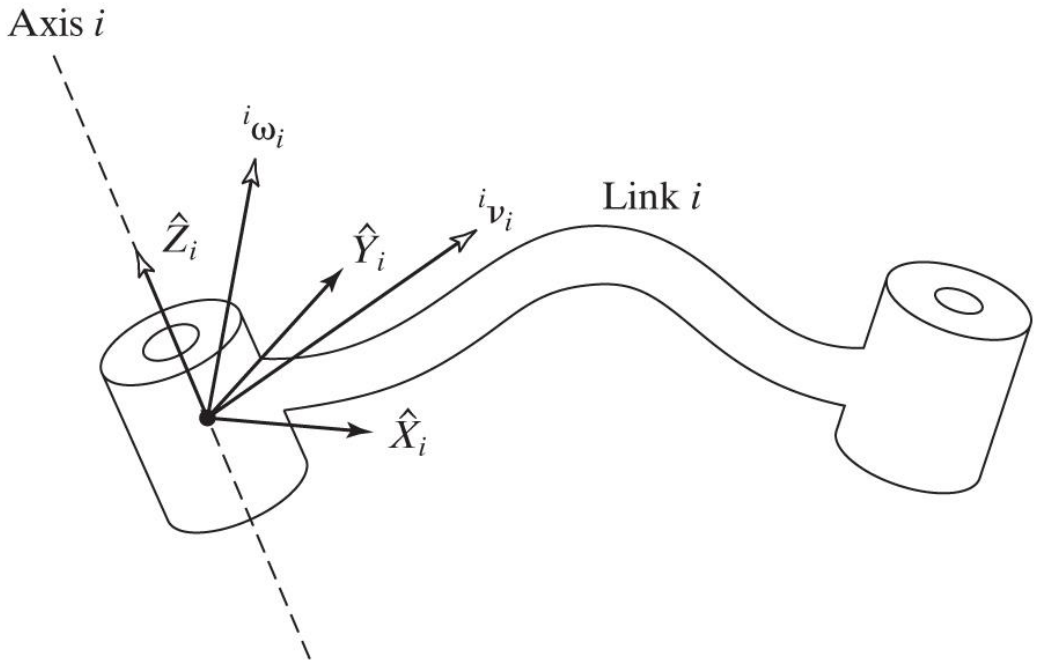
However, it is hard to assign physical meaning to this.

Velocity of Links

Consider the **frame assignments** given by **Modified DH Parameters**.

We can think of **each link in the robot as a rigid body** with:

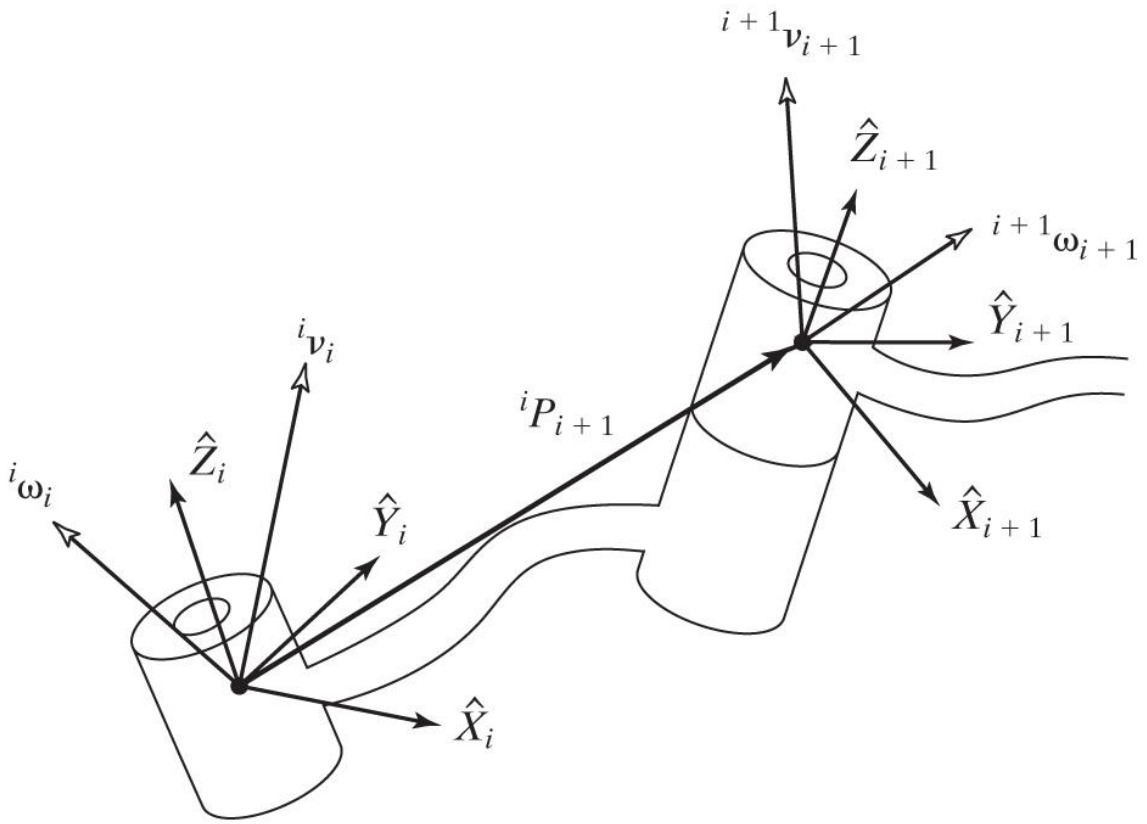
- **Angular velocity**, ${}^{i}\omega_i$
- **Linear velocity**, ${}^{i}v_i$



Since the robot is a **kinematic chain**,
the **velocity of link $i+1$** will be:

velocity of link i $+$ **velocity components due to joint $i+1$**

Velocity of Links – Revolute Joints (Exercise, 10 minutes)



For a **revolute joint, $i+1$**:

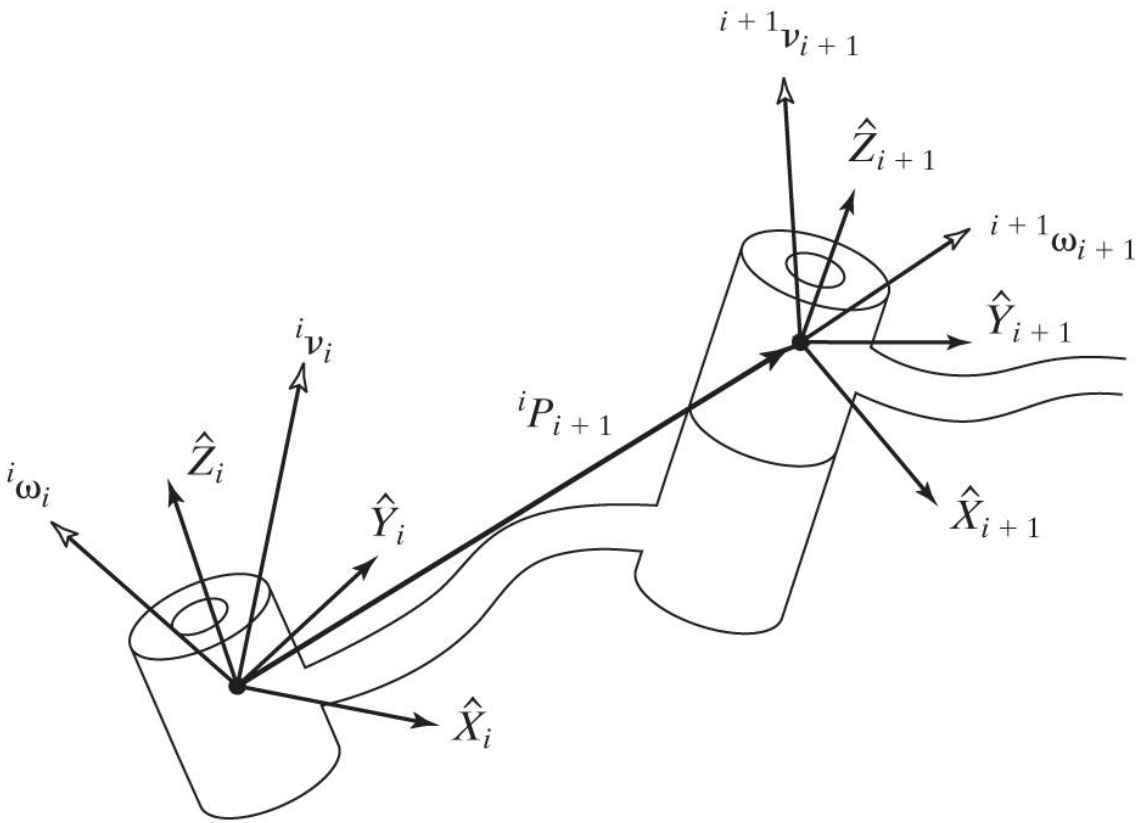
1. If we know the speed of rotation, $\dot{\theta}_{i+1}$, write the angular velocity vector arising from this rotation (*in frame* $i+1$).

2. Write an expression for $^{i}\omega_{i+1}$ (*assuming that we know* $^{i}\omega_{i}$).

3. Write an expression for $^{i}v_{i+1}$ (*assuming that we know* $^{i}v_{i}$).

4. Re-write 2. and 3. in reference frame $i+1$, i.e., calculate $^{i+1}\omega_{i+1}$ and $^{i+1}v_{i+1}$.

Velocity of Links (Results)



1. $^{i+1}\omega_{i+1,\dot{\theta}_{i+1}} = \dot{\theta}_{i+1}\,{}^{i+1}\hat{Z}_{i+1} = \begin{bmatrix} 0 \\ 0 \\ \dot{\theta}_{i+1} \end{bmatrix}$

2. $^{i}\omega_{i+1} = {}^{i}\omega_{i} + {}^{i}_{i+1}R\,\dot{\theta}_{i+1}\,{}^{i+1}\hat{Z}_{i+1}$

3. $^{i}v_{i+1} = {}^{i}v_{i} + {}^{i}\omega_{i} \times {}^{i}P_{i+1}$

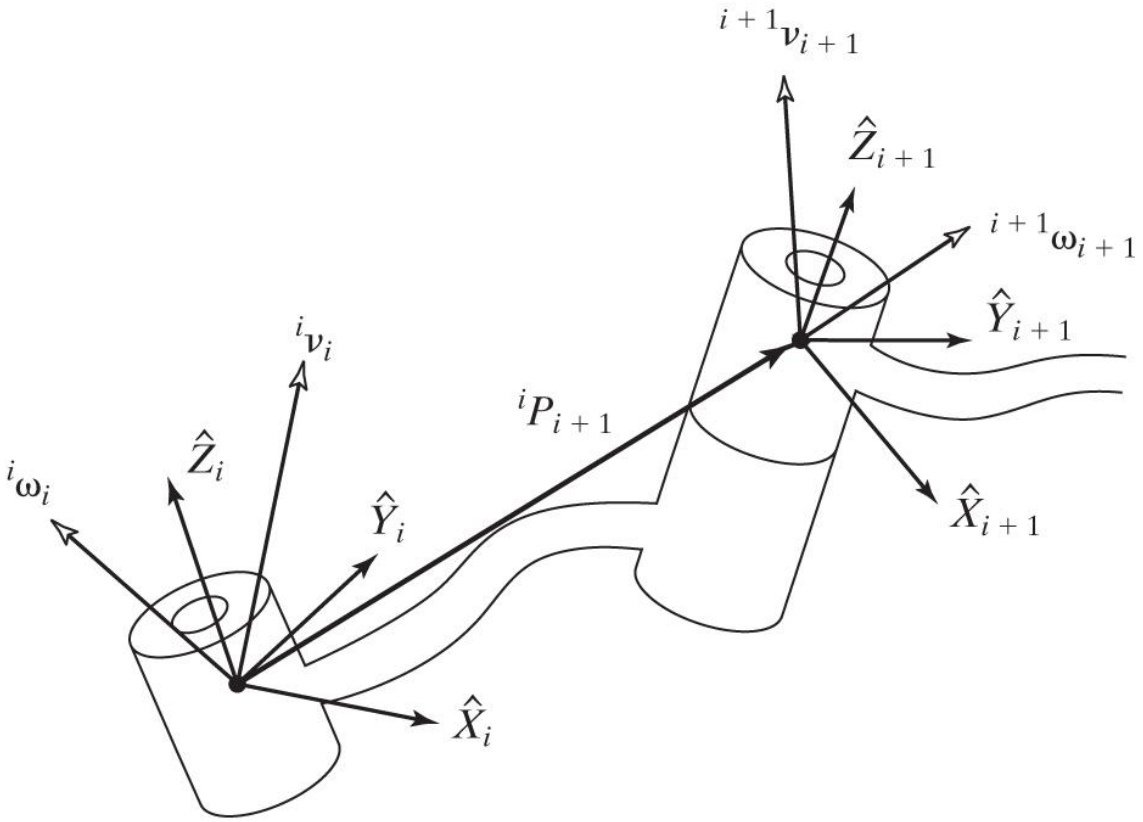
4. ${}^{i+1}_{i}R\ {}^{i}\omega_{i+1} = {}^{i+1}_{i}R({}^{i}\omega_{i} + {}^{i}_{i+1}R\,\dot{\theta}_{i+1}\,{}^{i+1}\hat{Z}_{i+1})$

$$^{i+1}\omega_{i+1} = {}^{i+1}_{i}R\ {}^{i}\omega_{i} + \dot{\theta}_{i+1}\,{}^{i+1}\hat{Z}_{i+1}$$

${}^{i+1}_{i}R\ {}^{i}v_{i+1} = {}^{i+1}_{i}R({}^{i}v_{i} + {}^{i}\omega_{i} \times {}^{i}P_{i+1})$

$$^{i+1}v_{i+1} = {}^{i+1}_{i}R({}^{i}v_{i} + {}^{i}\omega_{i} \times {}^{i}P_{i+1})$$

Velocity of Links – Prismatic Joints



The exact same logic follows for a **prismatic joint**:

$$^{i+1}\omega_{i+1} = {}^{i+1}_{i}R \; {}^{i}\omega_{i}$$

$$^{i+1}v_{i+1} = {}^{i+1}_{i}R(\; {}^{i}v_{i} + \; {}^{i}\omega_{i} \times \; {}^{i}P_{i+1}) + \dot{d}_{i+1}\,{}^{i+1}\hat{Z}_{i+1}$$

SDU ROBOTICS

Part II: Trajectory Generation

(for Linear Paths)

Offline vs. Online Robot Trajectory Control

Internally, **a robot has a control computer** that is updating the robot parameters and **sending commands** to the motors at a **fixed updated rate/control cycle**.

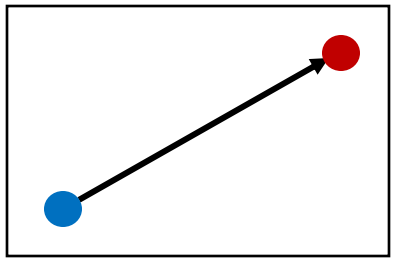
There are typically two ways for **a user** to control a robot:

- **Online control: Commands** are **sent** to the robot **at** its **update rate/control frequency**.
  - Example: UR e-series robots run at 500 Hz, so a new target point would have to be sent to the robot every 2 ms.
  - This means that **a path has to be discretized**/sampled to fit the robot's control cycle.

- **Offline control:** Specify a **start and end point**, a **type of motion**, and **constraints** on time/velocity.
  - Example: Move in a straight line in Cartesian space from $P_1 = \left[0.3, -0.2, 0.4, \frac{\pi}{2}, 0, 0\right]$ to $P_2 = \left[0.5, 0.1, 0.2, \frac{\pi}{2}, 0, 0\right]$
  - The robot internally figures out how to **translate this into an online control problem**.

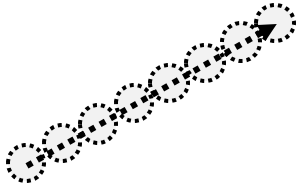
Offline vs. Online Robot Trajectory Control

Desired Path



Online Control

Discretized Path:



Offline Control

Move Type:

”Linear Cartesian Move”

or

”Linear Joint Move”

Parameters:

Start Point

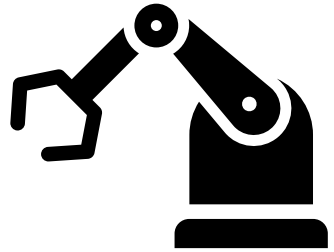
Target Point

Speed

Time

Send commands continuously

Send command once



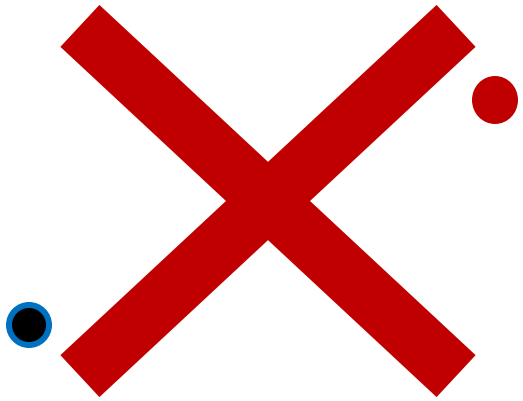
Robot Control Computer

(updates at certain control frequency)

SDU ROBOTICS

Getting from A to B: Multiple Paths





This is a linear path.
This is what we are discussing.

This path is not linear.
We are not considering this right now

<u>Linear paths will always take the shortest distance *in that particular space.*</u>

SDU ROBOTICS

Movement in Joint or Cartesian Space

We can **define trajectories** for a robot in **either Joint or Cartesian space**.

- **Joint space** has the advantage that we **do not have to consider singularities**.

- **Cartesian space** has the advantage that it is **easier to relate to the task** and to how we think.



Linear Movements: Cartesian ≠ Joint Space!

Linear movement in Cartesian space → curved movement in Joint Space!

Let us see it on the robot…

Getting from A to B: Multiple Velocity Profiles

All of these paths reach the target point at the same time:

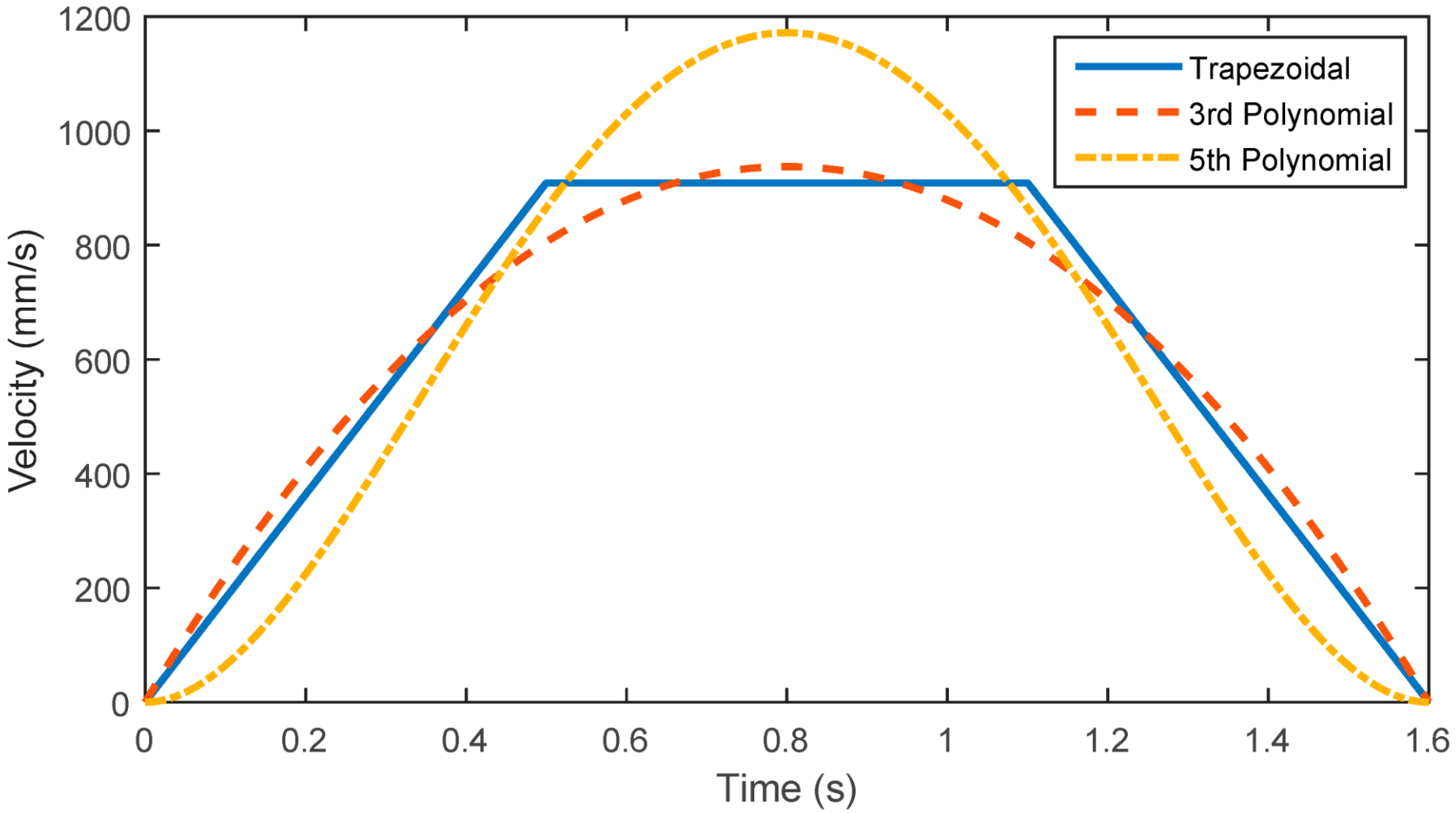


…but they do so with different velocity profiles.

This is what we will be discussing.

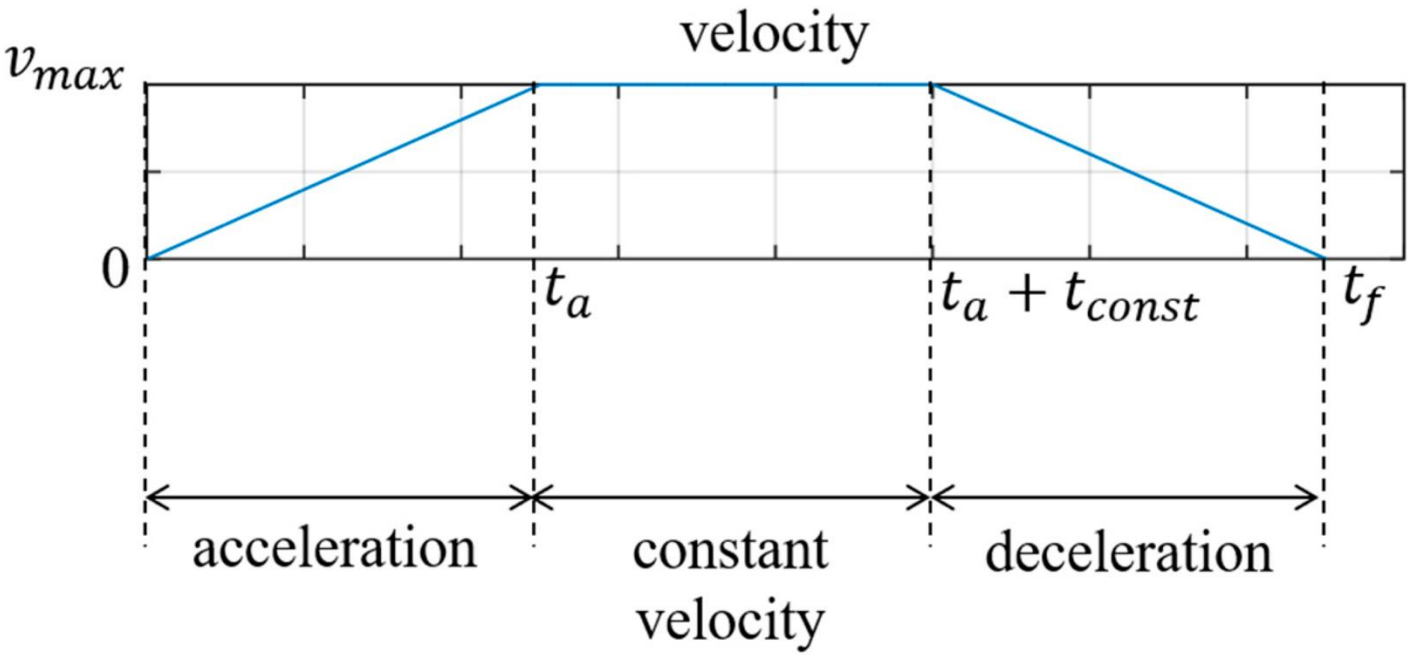
SDU ROBOTICS

Different Velocity Profiles



Source: Yoon et al. “Trapezoidal Motion Profile to Suppress Residual Vibration of Flexible Object Moved by Robot”, *Electronics*, MDPI, 2019, https://www.mdpi.com/2079-9292/8/1/30/htm

Trapezoidal Velocity Profiles



Specify either:

- Max. **velocity** and **acceleration**:

$$t_a = \frac{v_{max}}{a_{max}}$$

- Max. **velocity** and **time**:

$$a_{max} = \frac{v_{max}}{t_a}$$

- Max. **acceleration** and **time**:

$$v_{max} = a_{max} t_a$$

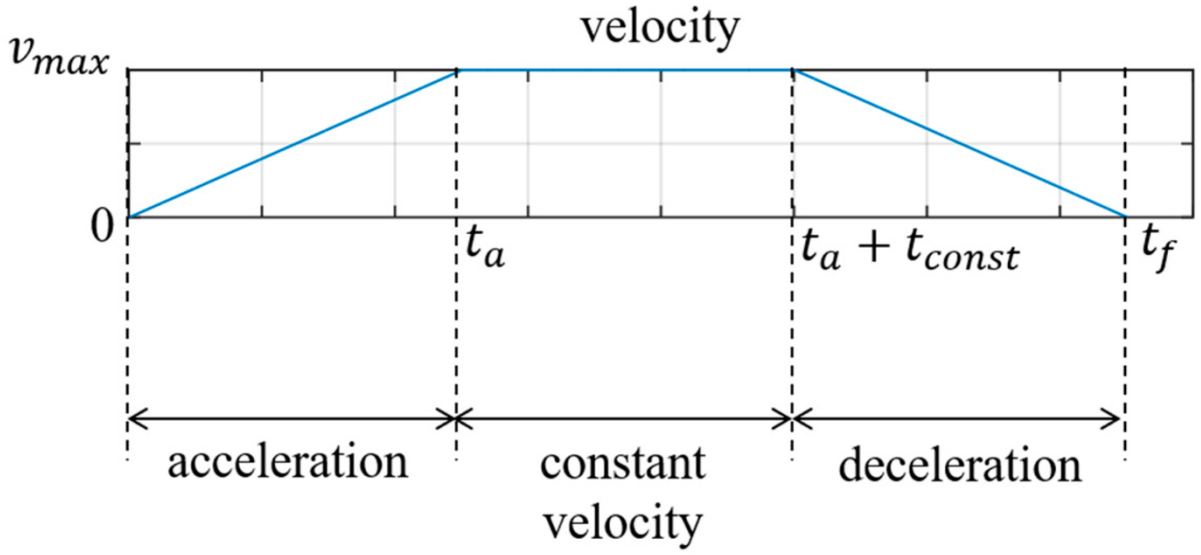
Then calculate:

$$t_f = \frac{\Delta x}{v_{max}} + t_a$$

$$t_{const} = t_f - 2t_a$$

Source: Yoon et al. “Trapezoidal Motion Profile to Suppress Residual Vibration of Flexible Object Moved by Robot”, *Electronics*, MDPI, 2019, https://www.mdpi.com/2079-9292/8/1/30/htm

Exercise (10 min.)



Given: $p_{start}, v_{max}, a_{max}, t_a, t_{const}$

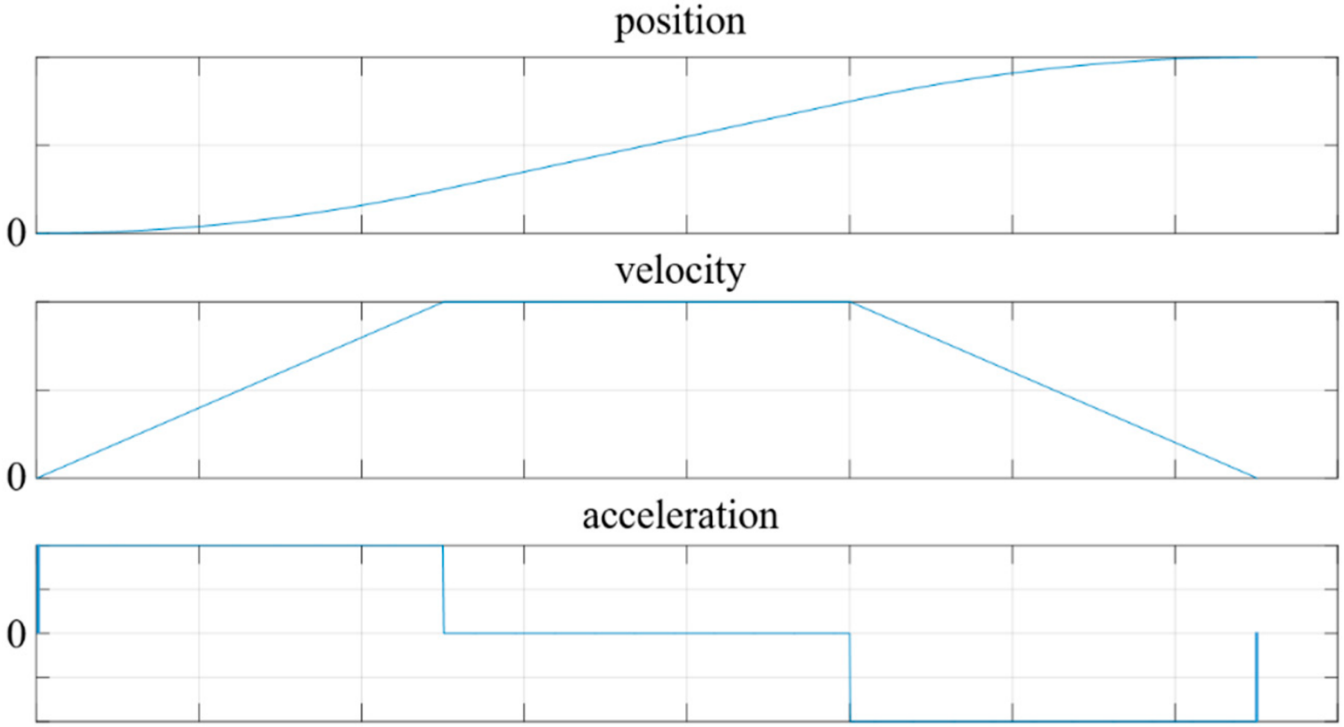
1. Write an equation to calculate the velocity at any given time, t for a trapezoidal velocity profile.
2. Do the same for position.

(Hint: They should be piecewise equations)

$$v(t) = \begin{cases} a_{max}t, & t < t_a \\ v_{max}, & t_a \leq t < t_a + t_{const} \\ v_{max} - a_{max}\left(t - (t_a + t_{const})\right), & t \geq t_a + t_{const} \end{cases}$$

$$p(t) = \begin{cases} \frac{1}{2}a_{max}t^2, & t < t_a \\ \frac{1}{2}a_{max}{t_a}^2 + v_{max}(t - t_a), & t_a \leq t < t_a + t_{const} \\ \frac{1}{2}a_{max}{t_a}^2 + v_{max}(t - t_a) - \frac{1}{2}a_{max}\left(t - (t_a + t_{const})\right)^2, & t \geq t_a + t_{const} \end{cases}$$

Trapezoidal Velocity Profiles: Derivatives



There is a problem…

Jerk is the **derivative of the acceleration**,
i.e., how fast the acceleration changes over time.

Why do we care about jerk?

- High jerk causes **wear in gears, motors and mechanical components**.
- In a car accident, **jerk** is what **causes damage to your neck from whiplash**.

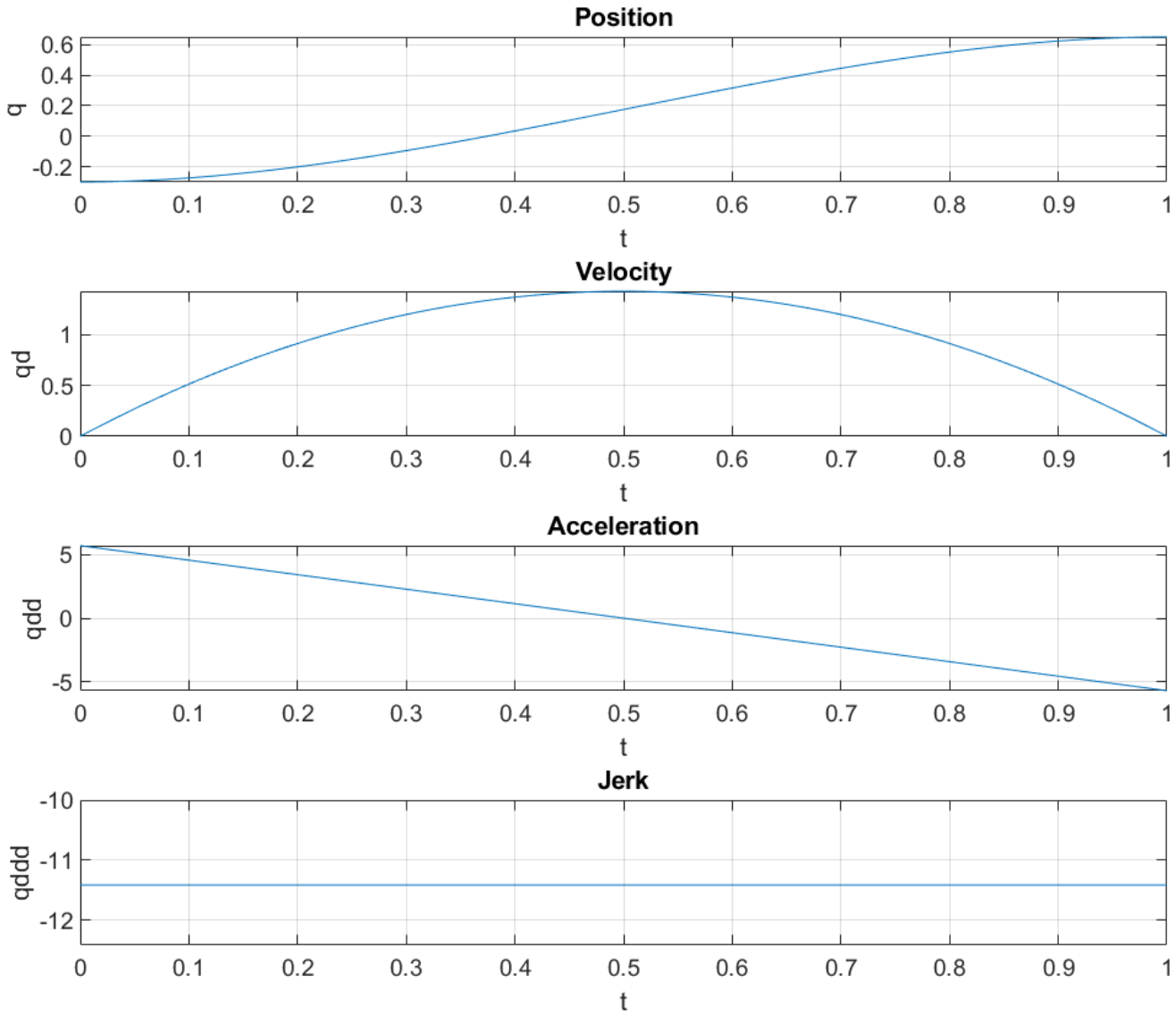
**For a trapezoidal velocity profile,
jerk is (theoretically) infinite!**

Cubic Polynomial

We can also specify a **trajectory** as a **cubic polynomial**:

$$q(t) = at^3 + bt^2 + ct + d$$
$$\dot{q}(t) = 3at^2 + 2bt + c$$
$$\ddot{q}(t) = 6at + 2b$$
$$\dddot{q}(t) = 6a$$

This will result in **smoother motion** than a trapezoidal velocity profile.



Boundary Constraints

To **calculate the parameters** of the cubic polynomial, we **set up constraints**:

- On the **starting point**:

$$q(0) = q_{start} = d$$
$$\dot{q}(0) = \dot{q}_{start} = c$$

- On the **end point**:

$$q(T) = q_{end} = aT^3 + bT^2 + cT + d$$
$$\dot{q}(T) = \dot{q}_{end} = 3aT^2 + 2bT + c$$

4 equations, 4 unknowns

Then we just **solve** for the parameters as a **system of equations**.

Higher Order Polynomials

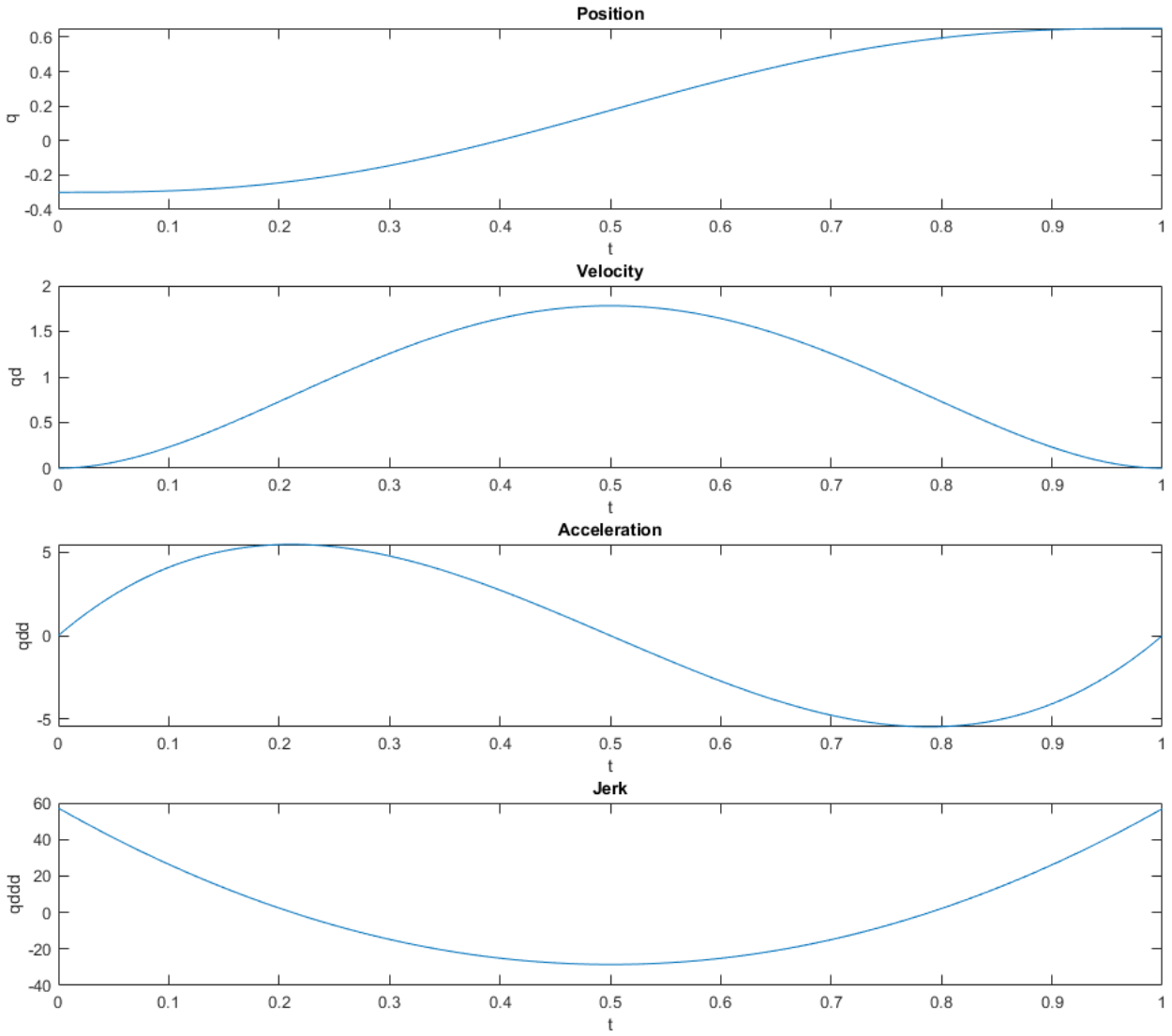
We can do the same for **higher-order polynomials.**

For example, a **fifth-order polynomial** allows us to specify **boundary constraints on the acceleration**:

$$q(t) = at^5 + bt^4 + ct^3 + dt^2 + et + f$$
$$\dot{q}(t) = 5at^4 + 4bt^3 + 3ct^2 + 2dt + e$$
$$\ddot{q}(t) = 20at^3 + 12bt^2 + 6ct + 2d$$

$$q(0) = q_{start} = f$$
$$\dot{q}(0) = \dot{q}_{start} = e$$
$$\ddot{q}(0) = \ddot{q}_{start} = 2d$$

$$q(T) = q_{end} = aT^5 + bT^4 + cT^3 + dT^2 + eT + f$$
$$\dot{q}(T) = \dot{q}_{end} = 5aT^4 + 4bT^3 + 3cT^2 + 2dT + e$$
$$\ddot{q}(T) = \ddot{q}_{end} = 20aT^3 + 12bT^2 + 6cT + 2d$$



SDU ROBOTICS

Trajectories for Multiple Joints/Dimensions

The previous examples consider only **one** joint/Cartesian space **dimension <u>independently</u>** at a time.

Generalizing to multiple dimensions is simply a matter of **making all parameters vectors**:

$$\boldsymbol{q}(t) = \boldsymbol{a}t^3 + \boldsymbol{b}t^2 + \boldsymbol{c}t + \boldsymbol{d}$$

$$\begin{bmatrix} q_1(t) \\ \vdots \\ q_n(t) \end{bmatrix} = \begin{bmatrix} a_1 \\ \vdots \\ a_n \end{bmatrix} t^3 + \begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix} t^2 + \begin{bmatrix} c \\ \vdots \\ c_n \end{bmatrix} t + \begin{bmatrix} d \\ \vdots \\ d_n \end{bmatrix}$$

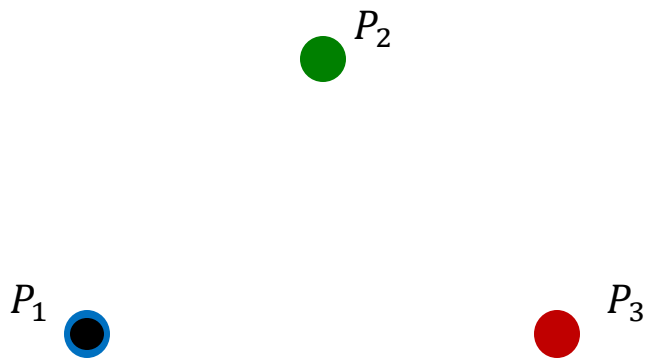
Question: The above is true for a complete joint-space position.
Will it work for a full Cartesian-space pose (position + orientation)?
Why or why not? If not, which part is it true for?
Discuss (5 minutes)

Linear Trajectories through Multiple Points

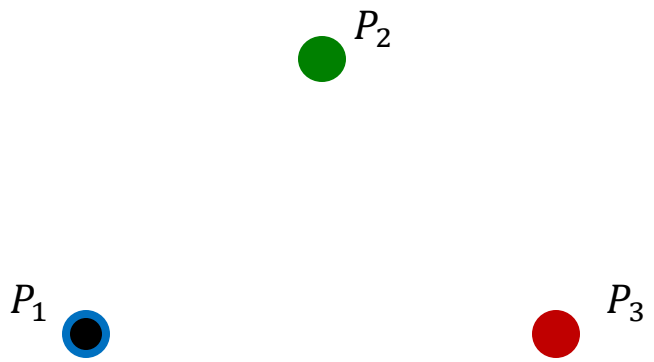
Question: with the tools so far, how do you obtain the following behavior?

(Discuss with your neighbors, 5 min., then share with the class)

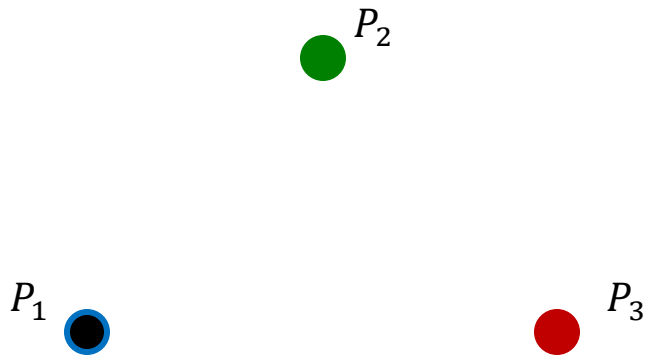
Stopping at middle point



Continuous motion through middle point

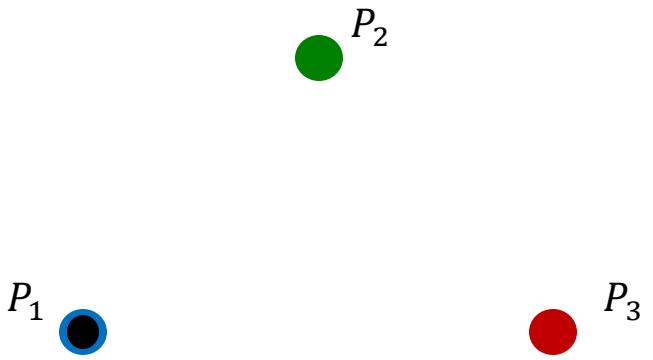


Linear Trajectories through Multiple Points



Two trapezoidal velocity profiles in a row:

- One from P_1 to P_2
- Another from P_2 to P_3



Two cubic polynomials:

$$q_1(t) = a_1t^3 + b_1t^2 + c_1t + d_1$$
$$\dot{q}_1(t) = 3a_1t^2 + 2b_1t + c_1$$
$$\ddot{q}_1(t) = 6a_1t + 2b_1$$

$$q_2(t) = a_2t^3 + b_2t^2 + c_2t + d_2$$
$$\dot{q}_2(t) = 3a_2t^2 + 2b_2t + c_2$$
$$\ddot{q}_2(t) = 6a_2t + 2b_2$$

with shared middle constraint:

$$q_1(0) = P_1 \qquad q_1(T_{end,1}) = q_2(0) = P_2 \qquad q_2(T_{end,2}) = P_3$$
$$\dot{q}_1(0) = 0 \qquad \dot{q}_1(T_{end,1}) = \dot{q}_2(0) = v \qquad \dot{q}_2(T_{end,2}) = 0$$

What about Cartesian-space Orientation?

The **methods** introduced so far **assume that each dimension is independent**.

When describing **orientation**, **all components are interdependent**.

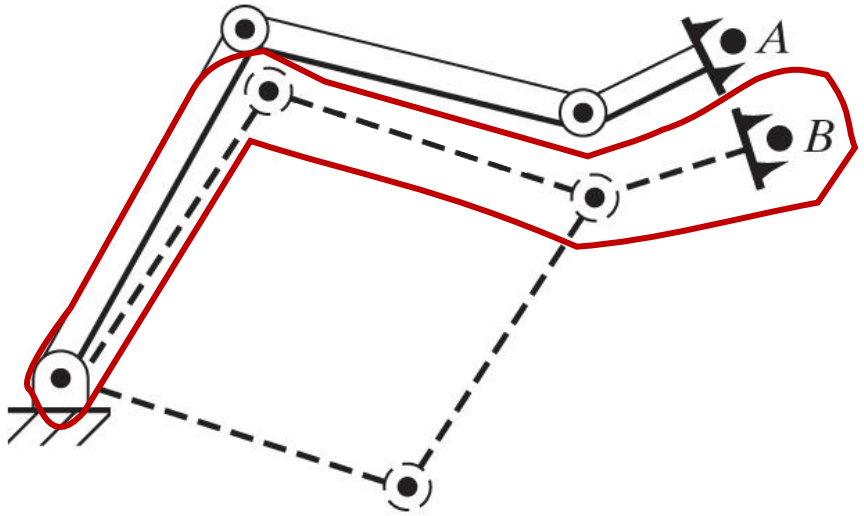
Orientations need to be treated separately using, e.g. quaternions.

This is much more complicated and beyond this course.

Moving in Cartesian Space: IK Solutions

When **moving in Cartesian space**, we also need to consider the existence of **multiple IK solutions**.

Within the same trajectory, we will usually want to pick the **closest solution to the current configuration**.



With redundant robots (and our own arm), we can **change solutions for the elbow while moving**.

Recap: What have we discussed today

- We can consider two types of **motion of a point or body: Linear** and **rotational velocity**
  - Rotational velocity can be expressed as **angular velocity**, a **skew-symmetric matrix** or the **derivative of Euler angles.**
- We can **propagate joint velocity from one link to another** in a robot.
- **Linear movements** in **Cartesian** and **Joint space** will always follow the **shortest distance** (**in that space!**)
- We can design different linear movements by changing the **velocity profile**:
  - **Trapezoidal**
  - **Polynomial** (3rd or 5th order)
- When **moving in Cartesian space**, we must always consider the **choice of IK solution**.

Take home message:

1. We can **propagate linear and rotational velocities down the kinematic chain** similarly to how we transform between frames.

2. For linear motions, **different velocity profiles** (e.g. **trapezoidal** or **polynomial**) will result in different features and degrees of smoothness. The **choice of profile** will influence our ability to **set boundary conditions**.

SDU ROBOTICS

After Easter…

The Jacobian Matrix: (perhaps the most important matrix in robotics)

→ **Relates motion of the joints with motion of the end-effector**

Concepts related to the Jacobian:

- **Numerical inverse kinematics** (conceptually)
- **Singularities**: theoretical understanding and practical consequences
- **Manipulability**: how close to a singularity is the robot? / how much force can it apply

Thank you for today.

Iñigo Iturrate

Ø27-604-3

inju@mmmi.sdu.dk